

의존성 역전의 사례와 해결

객체지향설계와 패턴
2026.03.24(Tue)



Dependency Inversion Principle

- 미리 정리 해보자
 - 주로 상속을 해주는 상위 클래스, 인터페이스, 추상 클래스 일수록 수정되거나 변하지 않는 게 당연하다.
 - 하위 클래스나 구체 클래스가 아닌 상위 클래스, 인터페이스, 추상 클래스를 통해 의존하라는 것이 바로 의존성 역전 원칙이다.
 - 간혹 저수준을 구현한 클래스를 사용하기 위해 사용하려는 클래스가 변경되지 않아야 한다는 것을 말한다.
- 클래스의 관계에서 말하지 않는 것
 - 어떤 클래스가 안 바뀌어야 하는 부분인가를 알아내는 게 중요하다.
 - 모두가 안 바뀌어야 하는 클래스들이라면 그들을 엮어줄 인터페이스가 필요하다.

일반적인 사례

- 우리가 보통 만들 게 되는 클래스들의 관계
- 외부 활동(outdoor)을 위해 주로 선글라스만 착용하는 경우가 대부분인데 보통 이런 정도의 코드를 작성하는 것이 일반적이다.
- 다음 코드를 보면 OutDoorWear 클래스가 SunGlass 클래스에 의존적이라고 볼 수 있다.

OutDoorWear에서 SunGlass를 착용

```
class SunGlass {  
    public void print() {  
        System.out.println("sunglass");  
    }  
}  
  
class OutDoorWear {  
    private SunGlass sunglass ;  
  
    public OutDoorWear() {  
        this.sunglass = new SunGlass ( ) ;  
    }  
    public void printWear () {  
        sunglass.print() ;  
    }  
}  
  
public class DIP_OutDoor {  
    public static void main(String[] args) {  
        OutDoorWear walking = new OutDoorWear() ;  
        walking.printWear();  
    }  
}
```

약간의 변경/수정 발생

- 그러다가 강한 햇빛을 가리기 위해 야구 모자를 착용한다고 해보자.



OutDoorWear에서 BaseballCap을 착용

```
class BaseballCap {  
    public void print() {  
        System.out.println("baseball cap");  
    }  
}  
  
class Sunglass {  
    public void print() {  
        System.out.println("sunglass");  
    }  
}  
  
class OutDoorWear {  
    // private Sunglass sunglass ;  
    private BaseballCap baseballCap ; // 이렇게 수정이 있어야 한다.  
    public OutDoorWear() {  
        // this.sunglass = new Sunglass ( ) ; //객체의 생성을 OutDoorWear에서 수행했다.  
        this.baseballCap = new BaseballCap() ;  
        // 저수준을 구현한 클래스를 사용하기 위해 여기가 수정되었다.  
    }  
}
```

OutDoorWear에서 BaseballCap을 착용

```
public void printWear () {  
    // sunglass.print() ;  
    baseballCap.print() ;  
    // 저수준을 구현한 클래스를 사용하기 위해 여기서도 수정이 필요하다.  
}  
}  
  
public class DIP_OutDoor {  
    public static void main(String[] args) {  
        OutDoorWear walking = new OutDoorWear() ;  
        walking.printWear();  
    }  
}
```

어색하고 이상한 변경과 수정

- 지금 알 수 있는 점은 저수준을 사용하는 클래스인 `OutdoorWear`가 바뀌었다는 점이다.
 - 선글라스 대신 야구모자만 착용하려는 것 뿐인데 `OutdoorWear`에서 수정이 있게 되었는데 이는 `OutdoorWear` 클래스가 구체적인 클래스(`SunGlass` 나 `BaseballCap`)에 의존하고 있다는 점이고 두 클래스간의 결합도(coupling)가 높아 유지보수가 힘들어질 수 있게 된다.
- 물론 두어개 클래스들이 이렇게 바뀐다면 찾아가며 수정해도 되긴 한데 개념상 저수준을 구현한 클래스를 사용하기 위해 사용하는 클래스가 바뀐다는 게 많이 이상하게 봐야 한다.

나은 방법은 무엇인가?

- 그렇다면 이런 이상한 내용을 어떻게 바꿔야 할까?
 - OutDoorWear가 구체적인 클래스인 Sunglass 나 BaseballCap이 아닌 추상화된 의상 클래스 또는 인터페이스 (Wearing)에만 의존하게 변경한다.
 - OutDoorWear는 착용하는 의상을 변경하더라도 OutDoorWear의 구조는 수정이나 변경의 영향을 받지 않도록 하자는 것이다.
- 반면 구체적인 클래스인 Sunglass나 BaseballCap 클래스들은 의상 인터페이스(Wearing)에 의존하게 되었는데 **의존 관계의 방향이 역전된 것**이다.

OutdoorWear와 Wearing

```
interface Wearing {  
    void print();  
}
```

```
class BaseballCap implements Wearing {  
    @Override  
    public void print() {  
        System.out.println("baseball cap");  
    }  
}
```

```
class SunGlass implements Wearing {  
    @Override  
    public void print() {  
        System.out.println("sunglass");  
    }  
}
```

// 새로 추가

```
class RainCoat implements Wearing {  
    @Override  
    public void print() {  
        System.out.println("raincoat");  
    }  
}
```

OutDoorWear와 Wearing

```
class OutDoorWear {  
    // private SunGlass sunglass ;  
    // private BaseballCap baseballCap ; // 이렇게 수정이 있어야 한다.  
    private Wearing wearing ; // 이렇게 바꾸자.  
  
    /*  
    public OutDoorWear() {  
        // this.sunglass = new SunGlass () ; // 객체의 생성을 OutDoorWear에서 수행했다.  
        //this.baseballCap = new BaseballCap() ;  
        // 저수준을 구현한 클래스를 사용하기 위해 여기가 수정되었다.  
    }  
    */  
    public OutDoorWear(Wearing wear) { // OutDoorWear() 생성자를 이렇게 바꾸자.  
        // 전달되는 매개변수(wear)를 이용하자. 단 생성은 호출쪽에서 해주는 걸로 하자.  
        this.wearing = wear ;  
    }  
  
    public void printWear () {  
        // sunglass.print() ;  
        // baseballCap.print() ;  
        // 저수준을 구현한 클래스를 사용하기 위해 여기도 수정이 필요하다.  
        wearing.print(); // 이렇게 해주면 되기도 한다.  
    }  
}
```

OutDoorWear와 Wearing

```
public class DIP_OutDoor {  
    public static void main(String[] args) {  
        // 생성자가 바뀌었으니 아래는 변경되어야 한다.  
        /*  
            OutDoorWear walking = new OutDoorWear() ;  
            walking.printWear();  
        */  
  
        OutDoorWear walking = new OutDoorWear (new SunGlass());  
        walking.printWear();  
        OutDoorWear restUnderParasol = new OutDoorWear (new BaseballCap());  
        restUnderParasol.printWear();  
        OutDoorWear inRaining = new OutDoorWear (new RainCoat());  
        inRaining.printWear();  
    }  
}
```

결론

- 좋아진건가?
 - 일단 여러가지의 의상을 착용해야 하는 경우라도 OutDoorWear의 수정/변경은 거의 없을 것이다.
 - ※ main() 이 있는 DIP_OutDoor 클래스는 테스트 드라이버일 뿐이니 “이거는 바뀌는데요.” 라고 하기 없기.
- 정리
 - 의존성 역전 원칙은 저수준이거나 구체적인 Sunglass, BaseballCap, RainCoat 클래스를 이용하기 위해 고정된 기능인 OutDoorWear 클래스가 변하지 않는 게 좋은 점이다.
 - 즉 고정된 기능을 갖는 클래스가 외부 내용을 사용하기 위해 구체 클래스에 의존하는 형태가 아닌 것이 좋다.